A Compiler for Lifting Packet-Processing Code to a Parallelism DSL
UGRC Proposal for CS4900 (June–Nov 2026)

Abhishek Gawai (CS23B020)
Guide: Dr. Srinivas N. G.
April, 2026

Background

Scaling network packet processing across multiple cores is a key challenge in modern networked systems. Many packet-processing applications are stateful, maintaining and updating memory regions across packets. Contention over shared memory across cores leads to cache bouncing and poor performance. Sharding avoids this by routing packets of the same flow to the same core, but suffers from load imbalance under skewed traffic. State-Compute Replication (SCR) is a recent technique that scales packet-processing throughput linearly with the number of cores, without synchronization, by replicating both state and computation across cores in a structured way.

Proposed Research Problem

To make SCR practical, it is valuable to automate the translation of sequential packet-processing C/C++ code into its SCR-parallel equivalent. Performing this translation manually is tedious and error-prone, as it requires identifying stateful data structures, extracting per-packet state transition logic, and restructuring the program to replay packet history across cores. This project proposes building a compiler, using Clang/LLVM, that automatically "lifts" sequential C/C++ packet-processing programs into a domain-specific language (DSL) that captures SCR parallelism abstractions, for example similar in spirit to the Casper compiler, which automatically retargets sequential Java programs to MapReduce frameworks using verified lifting. The project will focus on two components: (1) building Clang/LLVM compiler passes that analyze and translate C/C++ code into the DSL, and (2) testing the semantic equivalence of the translated DSL program against the original C/C++ code using manual and directed testing.

Proposed Timeline

- **Weeks 1–3: Literature Survey**
  - Study prerequisites and understand the SCR model.
  - Explore existing DSLs, such as Click, P4, etc., to evaluate their feasibility and determine if they can be adapted for the proposed project.
- **Weeks 4–5: Tooling Study**
  - Evaluate and select the optimal compilation framework (Clang vs. LLVM), followed by an in-depth study of the chosen toolchain.
  - Study the Casper compiler's architecture in detail.
  - Review the verified lifting approach used in Casper.
- **Weeks 6–9: Building the Compiler**

  - Implement Clang/LLVM passes to identify stateful data structures, extract per-packet state transition logic, and emit DSL representations.

- **Weeks 10–13: Semantic Equivalence Testing**
  - Validate the semantic equivalence of the translated DSL output against the original C/C++ program through manual and directed testing.
  - Summarize findings and implementation results, prepare a final report, and outline directions for extending the work.

References

[1] Qiongwen Xu, Sebastiano Miano, Xiangyu Gao, Tao Wang, Adithya Murugadass, Songyuan Zhang, Anirudh Sivaraman, Gianni Antichi, and Srinivas Narayana. State-Compute Replication: Parallelizing High-Speed Stateful Packet Processing. *USENIX NSDI*, 2025.

[2] Maaz Bin Safeer Ahmad and Alvin Cheung. Automatically Leveraging MapReduce Frameworks for Data-Intensive Applications. *ACM SIGMOD*, 2018. `https://casper-compiler.github.io/`